Aplicación
Métodos Numéricos

Jacobi · Gauss-Seidel · Newton-Raphson · Müller · Bairstow

Repositorio del proyecto (código fuente, guía y pruebas):

https://github.com/shadow025/metodos-jacobi-gauss-seidel

Elaborado por: **Luis Carlos Gómez**

Materia: Métodos Numéricos

Octubre de 2026

Aplicación de escritorio en Python que resuelve sistemas de ecuaciones lineales y encuentra raíces de ecuaciones y polinomios, mostrando la tabla de iteraciones y el procedimiento paso a paso de cada método.

Guía de uso

Esta guía explica cómo instalar y usar la aplicación. La aplicación tiene una ventana con cuatro pestañas, una por tipo de problema. En cada una se ingresan los datos, se presiona **Resolver** y se obtiene la tabla de iteraciones, el resultado y el procedimiento completo con las fórmulas sustituidas.

1. Qué incluye la aplicación

Método	Qué resuelve	Referencia
Jacobi	Sistemas de ecuaciones lineales Ax = b	Formulación clásica (Chapra & Canale, Burden & Faires)
Gauss-Seidel	Sistemas de ecuaciones lineales Ax = b, usando los valores ya actualizados en la misma iteración	Ídem
Newton-Raphson	Raíces de f(x) = 0 a partir de un valor inicial, con la derivada	Chapra & Canale, sección 6.2 (Ejemplo 6.3)
Müller	Raíces reales o complejas de f(x) = 0 a partir de tres puntos	Chapra & Canale, sección 7.2 (Ejemplo 7.2)
Bairstow	Todas las raíces (reales y complejas) de un polinomio	Chapra & Canale, sección 7.5 (Ejemplo 7.3)

2. Requisitos e instalación

Se necesita **Python 3.9 o superior**. La interfaz usa Tkinter, que viene incluido en el instalador oficial de python.org (en Linux puede requerir el paquete `python3-tk`).

1. Descargar o clonar el repositorio:

```
git clone https://github.com/shadow025/metodos-jacobi-gauss-seidel.git
cd metodos-jacobi-gauss-seidel
```

2. Crear (opcional) un entorno virtual e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate      # En Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

numpy *es la única dependencia obligatoria.* **sympy** *es opcional: si está instalado, Newton-Raphson deriva f(x) de forma simbólica; si no, usa derivada numérica.*

3. Ejecutar la aplicación:

```
python interfaz.py
```

3. La ventana principal

Al abrir la aplicación aparece una ventana con cuatro pestañas en la parte superior: **Jacobi / Gauss-Seidel**, **Newton-Raphson**, **Müller** y **Bairstow**. Todas tienen la misma organización:

- **Panel izquierdo:** los datos de entrada del método.
- **Cargar ejemplo:** llena los campos con el ejemplo de clase o del libro de Chapra & Canale, útil para comprobar que todo funciona.
- **Resolver:** ejecuta el método con los datos ingresados.
- **Panel derecho, sub-pestaña «Tabla de iteraciones»:** una fila por iteración con los valores intermedios y, debajo, el resumen (solución o raíz, si convergió y en cuántas iteraciones).
- **Panel derecho, sub-pestaña «Procedimiento paso a paso»:** el desarrollo completo como se haría a mano, con cada fórmula y los números sustituidos. El botón **Copiar** lo envía al portapapeles para pegarlo en un reporte.
- Si algún dato está mal escrito, aparece un mensaje indicando exactamente qué corregir; la aplicación no se cierra.

4. Pestaña Jacobi / Gauss-Seidel

Resuelve un sistema de n ecuaciones lineales con n incógnitas.

Campo	Qué ingresar	Ejemplo
Matriz A	Una fila por línea, con los coeficientes separados por espacio. Debe ser cuadrada y sin ceros en la diagonal.	20 1 -1 3 20 1 2 -3 20
Vector b	Los n términos independientes separados por espacio.	17 -18 25
x0	Vector inicial. Si se deja vacío se usa el vector de ceros.	0 0 0
Tolerancia	Se detiene cuando máx $\|x^{(k+1)} - x^{(k)}\|$ es menor que este valor.	1e-6
Máx. iteraciones	Límite de iteraciones si no se alcanza la tolerancia.	100
Método	Jacobi, Gauss-Seidel o Ambos (muestra las dos tablas para comparar).	Ambos

Qué muestra: la tabla tiene las columnas k, x1 … xn y el error de cada iteración. El resumen indica si la matriz es diagonalmente dominante (condición que garantiza la convergencia). El procedimiento muestra la verificación de dominancia fila por fila, los despejes de cada incógnita y, en cada iteración, cada incógnita con los valores sustituidos; en Gauss-Seidel marca con * los valores que ya se actualizaron en la misma iteración.

Si el programa avisa que hay un 0 en la diagonal o la matriz no es dominante y el método diverge, reordene las ecuaciones para que el coeficiente de mayor valor absoluto de cada fila quede en la diagonal.

5. Pestaña Newton-Raphson

Encuentra una raíz de f(x) = 0 con la fórmula $x_{i+1} = x_i - f(x_i) / f'(x_i)$.

Campo	Qué ingresar	Ejemplo
f(x)	La función, escrita con x como variable (ver sección 8).	exp(-x) - x
f'(x)	Opcional. Si se deja vacío la aplicación la calcula (simbólicamente con sympy o numéricamente). El resumen indica cuál usó.	-exp(-x) - 1
x0	Valor inicial.	0
Tolerancia εs (%)	Error relativo porcentual aproximado para detenerse: $\varepsilon a = \lvert(x_{i+1} - x_i) / x_{i+1}\rvert \cdot 100$.	0.0001
Máx. iteraciones	Límite de iteraciones.	100

Qué muestra: columnas i, x_i, $f(x_i)$, $f'(x_i)$, x_{i+1} y εa (%). El procedimiento escribe cada evaluación de f y f', la fórmula sustituida y el error de cada iteración, más la comprobación f(raíz). Si f'(x) = 0 en algún punto, el programa lo indica y pide otro valor inicial.

6. Pestaña Müller

Ajusta una parábola por tres puntos y toma como nueva aproximación la raíz de la parábola más cercana. Puede encontrar raíces complejas, que se muestran como a + bi.

Campo	Qué ingresar	Ejemplo
f(x)	La función (ver sección 8).	x^3 - 13x - 12
x0, x1, x2	Los tres puntos iniciales; deben ser distintos entre sí.	4.5, 5.5, 5
Tolerancia εs (%)	Error relativo porcentual aproximado para detenerse.	0.0001
Máx. iteraciones	Límite de iteraciones.	100

Qué muestra: en cada iteración los tres puntos, la nueva aproximación x3 y εa (%). El procedimiento detalla h0, h1, δ0, δ1, a, b, c, el discriminante, la elección del denominador de mayor magnitud y x3. Para probar una raíz compleja: f(x) = x^2 + 4 con x0 = 0, x1 = 1, x2 = 2 da 2i.

7. Pestaña Bairstow

Encuentra todas las raíces de un polinomio dividiéndolo entre factores cuadráticos $x^2 - r \cdot x - s$ y corrigiendo r y s hasta que el residuo sea cero.

Campo	Qué ingresar	Ejemplo
Coeficientes	Del grado mayor al menor, separados por espacio. Si un término no existe se escribe 0. Debajo se muestra el polinomio interpretado para verificar.	1 -3.5 2.75 2.125 -3.875 1.25 ($x^5 - 3.5x^4 + 2.75x^3 + 2.125x^2 - 3.875x + 1.25$)
r inicial, s inicial	Valores iniciales del factor cuadrático.	-1, -1
Tolerancia εs (%)	Se exige que $\|\Delta r/r\| \cdot 100$ y $\|\Delta s/s\| \cdot 100$ bajen de este valor.	1
Máx. iteraciones	Límite de iteraciones por factor.	100

Qué muestra: una fila por iteración de cada factor con r, s, Δr, Δs y los errores. El resumen lista cada factor cuadrático con sus dos raíces y, al final, todas las raíces del polinomio. El procedimiento escribe la división sintética (coeficientes b_i y c_i), el sistema 2×2 para Δr y Δs, su solución, las raíces del factor con la fórmula cuadrática y el polinomio reducido con el que se continúa.

8. Cómo escribir funciones

En Newton-Raphson y Müller la función se escribe como texto, usando **x** como variable. Se aceptan estas formas:

Operación	Se escribe	Ejemplo completo
Potencia	x^3 o x**3	x^3 - 2x - 5
Producto	13*x o 13x	x^3 - 13x - 12
Exponencial	exp(x)	exp(-x) - x
Logaritmo natural / base 10	log(x) o ln(x) / log10(x)	ln(x) - 1
Raíz cuadrada	sqrt(x)	sqrt(x) - 2
Trigonométricas (radianes)	sin(x) o sen(x), cos(x), tan(x)	cos(x) - x
Valor absoluto	abs(x)	abs(x) - 1
Constantes	pi, e	sin(pi*x)

9. Ejemplos listos y pruebas

Además de la interfaz, el repositorio incluye scripts de consola y pruebas automáticas:

Comando	Qué hace
python interfaz.py	Abre la aplicación gráfica (uso recomendado).
python ejemplo.py	Resuelve el sistema 3×3 de clase con Jacobi y Gauss-Seidel e imprime las tablas.
python ejemplo_raices.py	Resuelve los ejemplos 6.3, 7.2 y 7.3 de Chapra & Canale con Newton-Raphson, Müller y Bairstow.
python interactivo.py	Modo consola para ingresar un sistema lineal por teclado.
python -m unittest discover -s tests	Ejecuta las 22 pruebas unitarias; debe terminar con OK.

10. Problemas comunes

Mensaje o síntoma	Causa	Solución
No module named 'numpy'	No se instalaron las dependencias.	pip install -r requirements.txt
No module named 'tkinter'	El Python instalado no trae Tkinter (Linux).	Instalar python3-tk (Debian/Ubuntu: sudo apt install python3-tk).
Tiene un 0 en la diagonal	Alguna ecuación tiene $a_{ii} = 0$.	Reordenar las filas del sistema.
NO convergió (Jacobi / Gauss-Seidel)	La matriz no es diagonalmente dominante.	Reordenar las ecuaciones o aumentar las iteraciones.
Nombre desconocido en la expresión	Se escribió una función que no existe (p. ej. cosx sin paréntesis).	Revisar la tabla de la sección 8.
f'(x) = 0 (Newton-Raphson)	El valor inicial cae en un punto de pendiente cero, o el método entra en un ciclo.	Cambiar x0.
Los tres puntos iniciales deben ser distintos (Müller)	Dos de x0, x1, x2 son iguales.	Usar tres valores distintos.
El sistema para Δr y Δs es singular (Bairstow)	Los r, s iniciales producen un sistema sin solución.	Cambiar r y s iniciales (por ejemplo -1 y -1).

11. Estructura del repositorio

Archivo	Contenido
interfaz.py	Interfaz gráfica (Tkinter) con las cuatro pestañas.
metodos_iterativos.py	Funciones jacobi(), gauss_seidel() y es_diagonalmente_dominante().
metodos_raices.py	Funciones newton_raphson(), muller(), bairstow() y crear_funcion().
procedimiento.py	Genera el procedimiento paso a paso en texto para cada método.
ejemplo.py, ejemplo_raices.py, interactivo.py	Scripts de consola con los ejemplos y el modo interactivo.
tests/	Pruebas unitarias de los cinco métodos y del procedimiento.
README.md, GUIA_DE_USO.md	Documentación del proyecto.

Repositorio: https://github.com/shadow025/metodos-jacobi-gauss-seidel